

TASK 04 — Distributed Tracing, Correlation IDs & Structured Logging

Step 1 — Understand distributed tracing

What is Distributed Tracing?

Distributed tracing is a technique used to follow a single request as it travels through multiple microservices.

Example:

User

| Request

API Gateway

| Trace/Correlation ID = ABC-123456

Order Service

| Trace/Correlation ID = ABC-123456

Payment Service

| Trace/Correlation ID = ABC-123456

Notification Service

The same request can therefore be identified across the entire system.

1. Trace ID

A **Trace ID** identifies the complete request journey.

Example: Trace ID = 8f3a7c91

The same Trace ID can be associated with:

Gateway

↓

Order Service

↓

Payment Service

↓

Notification Service

Trace ID = ID of the entire request journey.

2. Span

A **Span** represents one individual operation inside a distributed request.

For example:

Trace

|— Gateway Span

|— Order Service Span

|— Payment Service Span

|— Notification Service Span

Suppose the complete request takes 2 seconds:

Trace ID: ABC123

Gateway Span → 100 ms

Order Span → 500 ms

Payment Span → 900 ms

Notification Span → 200 ms

You can determine which service consumed the most time.

Span = one operation within a trace.

3. Correlation ID

A **Correlation ID** is an identifier used to connect logs belonging to the same request.

Example: X-Correlation-ID: ABC-123456

The Gateway creates or receives this ID.

Then it sends the same ID to downstream services.

Gateway

Correlation ID: ABC-123456

↓

Order Service

Correlation ID: ABC-123456

↓

Payment Service

Correlation ID: ABC-123456



Notification Service

Correlation ID: ABC-123456

Correlation ID = common ID used to connect logs from different services.

4. Distributed Request

A **distributed request** is a request that travels through multiple services.

For example, when you purchase a product:

POST /orders



API Gateway



Order Service



Payment Service



Notification Service

One user action has caused several backend operations.

This is why distributed tracing is important in microservices.

5. Structured Logging

Instead of writing logs like:

Payment failed

we can write structured logs:

```
{  
  "timestamp": "2026-10-08T09:30:00",  
  "service": "payment-service",
```

```
"traceId": "ABC-123456",  
"spanId": "PAY-001",  
"message": "Payment failed"  
}
```

Now we can search for:

traceId = ABC-123456

and find all logs belonging to that request.

Step 2 — Create Correlation ID at Gateway

X-Correlation-ID: ABC-123456

The API Gateway should check whether the incoming request already contains a correlation ID.

Case 1 — ID exists

Client sends:

GET /orders

X-Correlation-ID: ABC-123456

Gateway keeps the same ID.

Case 2 — ID does not exist

Client sends:

GET /orders

Gateway generates a new ID:

X-Correlation-ID: 7f9a2c10

Then the Gateway passes it to the next service.

Step 3 — Forward the ID to Every Service

Correlation ID

ABC-123456



API Gateway

| X-Correlation-ID:

| ABC-123456



Order Service

| X-Correlation-ID:

| ABC-123456



Payment Service

| X-Correlation-ID:

| ABC-123456



Notification Service

All services should preserve the correlation ID.

Gateway → ABC-123456

Order Service → ABC-123456

Payment Service → ABC-123456

Notification → ABC-123456

Add the ID to Logs

Each service should include the correlation ID in its logs.

Gateway

[ABC-123456] Request received

Order Service

[ABC-123456] Creating order

Payment Service

[ABC-123456] Processing payment

Notification Service

[ABC-123456] Sending order confirmation

Now if the payment fails, search:

ABC-123456

You can see the complete request flow.

Example Complete Flow

Suppose the customer creates an order.

1. Client → Gateway

POST /orders

Gateway creates:

X-Correlation-ID: ORD-10001

2. Gateway → Order Service

POST /orders

X-Correlation-ID: ORD-10001

Order Service logs:

ORD-10001 - Creating order

3. Order Service → Payment Service

POST /payments

X-Correlation-ID: ORD-10001

Payment Service logs:

ORD-10001 - Processing payment

4. Payment Service → Notification Service

POST /notifications

X-Correlation-ID: ORD-10001

Notification Service logs:

ORD-10001 - Sending confirmation

What Happens When an Error Occurs

Suppose Payment Service fails.

Gateway



Order Service



Payment Service 



Payment failed

Logs might look like:

Gateway:

[ORD-10001] Request received

Order Service:

[ORD-10001] Order created

Payment Service:

[ORD-10001] Payment processing started

Payment Service:

[ORD-10001] Payment failed

You simply search:

ORD-10001

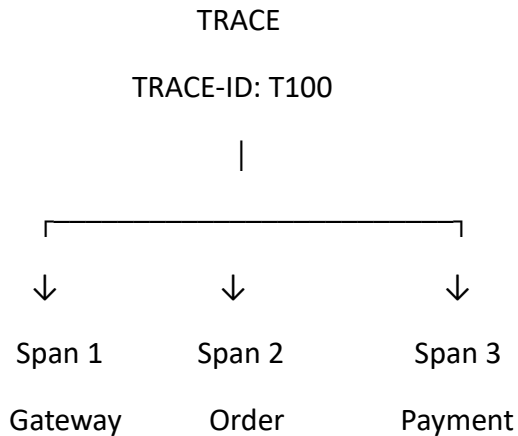
and find the complete request history.

Correlation ID vs Trace ID vs Span ID

Term	Meaning
Correlation ID	Connects logs belonging to the same request
Trace ID	Identifies the complete distributed request

Term	Meaning
Span ID	Identifies one operation/service within a trace
Span	Represents one operation
Structured Logging	Logs data in a consistent machine-readable format

Think of it like this:



Practical Spring Boot Implementation

Create a Correlation ID Filter

@Component

```

public class CorrelationIdFilter extends OncePerRequestFilter {

    private static final String CORRELATION_ID = "X-Correlation-ID";

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String correlationId = request.getHeader(CORRELATION_ID);
        if (correlationId == null || correlationId.isBlank()) {

```

```

        correlationId = UUID.randomUUID().toString();
    }

    response.setHeader(CORRELATION_ID, correlationId);
    MDC.put("correlationId", correlationId);
    try {
        filterChain.doFilter(request, response);
    } finally {
        MDC.remove("correlationId");
    }
}
}

```

Add Correlation ID to Logs

Configure your logging pattern.

For example:

```
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%X{correlationId}] %-5level %logger - %msg%n
```

Now logs can appear as:

```
2026-10-08 09:30:10 [ABC-123456] INFO OrderService - Creating order
```

```
2026-10-08 09:30:11 [ABC-123456] INFO PaymentService - Processing payment
```

```
2026-10-08 09:30:12 [ABC-123456] ERROR PaymentService - Payment failed
```

The important part is:

```
[ABC-123456]
```

Forward Correlation ID Between Services

Suppose Order Service calls Payment Service using RestClient.

```
RestClient restClient = RestClient.builder()
```

```
    .baseUrl("http://localhost:8082")
```

```
.build();
```

You can forward the correlation ID:

```
String correlationId = MDC.get("correlationId");
```

```
String response = restClient.post()
```

```
.uri("/payments")
```

```
.header("X-Correlation-ID", correlationId)
```

```
.body(paymentRequest)
```

```
.retrieve()
```

```
.body(String.class);
```

Therefore:

Order Service

| X-Correlation-ID: ABC-123456

Payment Service

Step 4 — Add Structured Logs

What is a structured log

A structured log stores information in a consistent format, usually **JSON**.

Instead of writing:

Payment failed

we store detailed information:

```
{  
  "timestamp": "2026-10-08T09:30:10",  
  "service": "payment-service",  
  "traceId": "ABC-123456",  
  "endpoint": "/payments",  
  "method": "POST",  
  "status": 500,
```

```
"duration": 1200  
}
```

This makes it much easier to search and troubleshoot problems.

Every log should contain:

1. Timestamp

Shows **when the request happened**.

timestamp: 2026-10-08 09:30:10

Useful for finding when an issue occurred.

2. Service

Shows **which microservice generated the log**.

service: payment-service

For example:

gateway-service

order-service

payment-service

notification-service

3. Trace / Correlation ID

Identifies the request across all services.

traceId: ABC-123456

For example:

Gateway → ABC-123456

Order Service → ABC-123456

Payment Service → ABC-123456

Notification → ABC-123456

This is extremely important for distributed tracing.

4. Endpoint

Shows which API was called.

endpoint: /api/payments

Example:

POST /api/payments

5. HTTP Method

Shows what type of HTTP operation was performed.

Common methods:

GET

POST

PUT

PATCH

DELETE

Example:

method: POST

6. Status

Shows the result of the request.

Successful:

status: 200

Created:

status: 201

Client error:

status: 400

Not found:

status: 404

Server error:

status: 500

7. Duration

Shows how long the request took.

Example:

duration: 850 ms

This helps identify slow services.

For example:

Order Service → 100 ms

Payment Service → 2500 ms

Notification → 200 ms

We can immediately see that **Payment Service is slow**.

Example Structured Logs

Gateway

```
{  
  "timestamp": "09:30:00",  
  "service": "gateway",  
  "traceId": "ABC-123456",  
  "endpoint": "/orders",  
  "method": "POST",  
  "status": 200,  
  "duration": 50  
}
```

Order Service

```
{  
  "timestamp": "09:30:01",  
  "service": "order-service",  
  "traceId": "ABC-123456",
```

```
"endpoint": "/orders",  
"method": "POST",  
"status": 201,  
"duration": 150  
}
```

Payment Service

```
{  
  "timestamp": "09:30:02",  
  "service": "payment-service",  
  "traceId": "ABC-123456",  
  "endpoint": "/payments",  
  "method": "POST",  
  "status": 200,  
  "duration": 400  
}
```

All three logs have:

traceId = ABC-123456

Therefore, we know they belong to the same request.

Step 5 — Test a Successful Request

Now we test the system normally.

Suppose the user creates an order.

Client



API Gateway



Order Service



Payment Service



Notification Service

The request receives:

Trace ID = ABC-123456

Then the ID travels through every service.

Gateway

[ABC-123456] Request received

Status: 200

Order Service

[ABC-123456] Order created

Status: 201

Payment Service

[ABC-123456] Payment successful

Status: 200

Notification Service

[ABC-123456] Notification sent

Status: 200

Finally: Request → SUCCESS

What are we testing

We are checking that:

- The request reaches the Gateway.
- Gateway forwards the ID.
- Order Service receives the ID.
- Payment Service receives the ID.

- Notification Service receives the ID.
- All services generate logs.
- The request completes successfully.

Step 6 — Create an Intentional Failure

Payment Service → unavailable

For example:

Gateway



Order Service



Payment Service ❌



Notification Service

We can temporarily stop Payment Service.

Example:

`docker stop payment-service`

or if running Spring Boot directly, stop the Payment Service application.

Now try creating an order again.

POST /orders

The Gateway creates:

Trace ID = XYZ-789456

The request travels:

Gateway



Order Service



Payment Service ❌

Payment Service is unavailable.

The user might receive:

HTTP 503 Service Unavailable

Step 7 — Trace the Failed Request

1. Where did the request start

Search for the Trace ID:

XYZ-789456

You might find:

Gateway:

[XYZ-789456] POST /orders

This tells us:

The request started at the API Gateway.

2. Which service failed

Continue searching for:

XYZ-789456

You might see:

Gateway → SUCCESS

Order Service → SUCCESS

Payment Service → FAILED

Therefore:

Payment Service is the failed service.

3. What is the error message

The Payment Service might produce:

Connection refused

or:

Payment Service unavailable

or:

Connection timeout

For example:

```
{  
  "traceId": "XYZ-789456",  
  "service": "payment-service",  
  "status": 503,  
  "error": "Payment service unavailable"  
}
```

Now we know exactly what went wrong.

4. What was the response time

Suppose the logs show:

Gateway → 100 ms

Order Service → 200 ms

Payment Service → 3000 ms

This tells us Payment Service took **3 seconds** before failing.

Therefore, we can investigate:

- Network connection
- Service availability
- Timeout configuration
- Database connection
- External payment provider

Complete Failure Example

Suppose we have:

Trace ID = XYZ-789456

The logs are:

09:35:00 Gateway

[XYZ-789456] POST /orders

Status: 200

Duration: 50 ms

09:35:01 Order Service

[XYZ-789456] Creating order

Status: 201

Duration: 150 ms

09:35:04 Payment Service

[XYZ-789456] Calling payment provider

Status: 503

Duration: 3000 ms

09:35:04 Gateway

[XYZ-789456] Payment service unavailable

Status: 503

From this information we can immediately understand:

Request started:

↓

API Gateway

↓

Order Service

↓

Payment Service ❌

↓

HTTP 503

Successful vs Failed Request

Successful request

Client



Gateway ✓



Order ✓



Payment ✓



Notification ✓



Response 200

Failed request

Client



Gateway ✓



Order ✓



Payment ✗



Response 503

The **Trace/Correlation ID** allows us to connect all these logs.

Step 8 — Study Distributed Tracing Tools / Concepts

1. OpenTelemetry

[OpenTelemetry](#)

OpenTelemetry (OTel) is a widely used open-source observability framework.

It can collect:

- Traces
- Metrics
- Logs

For distributed tracing, OpenTelemetry creates and propagates **Trace IDs and Span IDs**.

Example

Trace ID: ABC-123456

Gateway

└─ Span: 1001

Order Service

└─ Span: 1002

Payment Service

└─ Span: 1003

Notification Service

└─ Span: 1004

OpenTelemetry is mainly the **instrumentation and telemetry standard/toolkit**. It does not have to be the place where you visualize the traces.

2. Jaeger

[Jaeger](#)

Jaeger is a distributed tracing platform used to collect, store, and visualize traces.

A typical flow is:

Spring Boot Services



OpenTelemetry



Jaeger



Jaeger UI

In the Jaeger UI, you can see:

Trace: ABC-123456

Gateway ————— 100 ms

Order Service ————— 300 ms

Payment Service ————— 800 ms

Notification — 100 ms

This makes it easy to identify the slow service.

3. Zipkin

[Zipkin](#)

Zipkin is another distributed tracing system.

It helps answer questions such as:

- Where did the request start?
- Which services did it visit?
- Which service was slow?
- Where did the request fail?

Example:

Client



Gateway 50 ms



Order 150 ms



Payment 900 ms ← Slow



Notification 100 ms

4. Grafana Tempo

[Grafana Tempo](#)

Grafana Tempo is a distributed tracing backend designed to work well with the Grafana observability ecosystem.

Typical architecture:

Application



OpenTelemetry



Tempo



Grafana

Grafana can be used to view traces alongside metrics and logs.

5. AWS X-Ray

[AWS X-Ray](#)

AWS X-Ray is an AWS service for analyzing distributed applications.

It is useful when your microservices are running on AWS.

Example:

API Gateway



Lambda / EC2



DynamoDB



External Service

X-Ray helps visualize the request path and identify latency and errors.

6. Datadog APM

[Datadog APM](#)

Datadog APM provides application performance monitoring and distributed tracing.

It can show:

- Request latency
- Service dependencies
- Errors
- Trace information
- Database calls

Example:

Order Service



Payment Service



Payment Database

You can investigate the entire transaction.

7. New Relic

[New Relic](#)

New Relic provides application monitoring and distributed tracing.

It can help identify:

Slow API



Slow Microservice



Slow Database Query

This is useful for production troubleshooting.

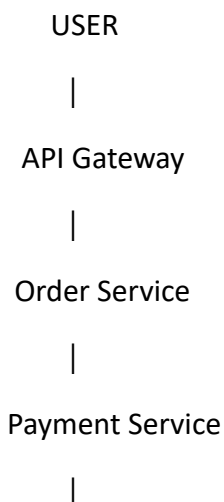
Comparison

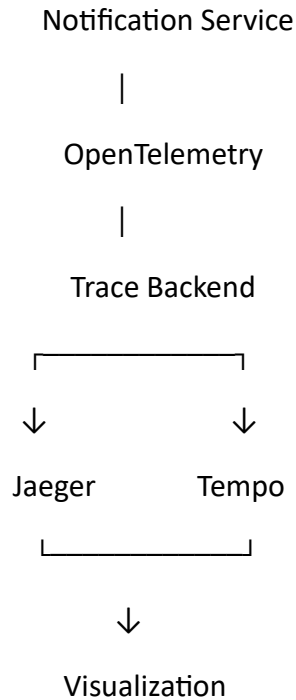
Tool	Main Purpose	Typical Use
OpenTelemetry	Instrumentation/telemetry	Generate and propagate traces
Jaeger	Trace collection & visualization	View distributed traces
Zipkin	Distributed tracing	Trace microservice requests
Grafana Tempo	Trace backend	Store/query traces with Grafana
AWS X-Ray	AWS distributed tracing	AWS applications
Datadog APM	Monitoring + tracing	Production applications
New Relic	Monitoring + tracing	Application performance

How They Work Together

A very important concept is that you don't necessarily choose only one tool.

For example, with a Spring Boot microservices application:





The important distinction is:

OpenTelemetry → collects/instruments telemetry

Jaeger/Tempo/etc. → store/query/visualize traces

When studying distributed tracing tools, focus on these terms:

1. Trace ID

Identifies the complete request.

ABC-123456

2. Span ID

Identifies an individual operation.

Gateway Span → 001

Order Span → 002

Payment Span → 003

3. Parent-Child Relationship

Shows how operations are connected.

Gateway Span

└─ Order Span

└─ Payment Span

4. Context Propagation

Carries tracing information from one service to another.

Gateway

↓ Trace Context

Order

↓ Trace Context

Payment

5. Sampling

Controls how many traces are collected.

Example:

1,000 requests

↓

10% sampling

↓

100 traces collected

This helps control storage and processing costs.

What is OpenTelemetry

OpenTelemetry (OTel) is an open-source observability framework used to collect:

- **Traces**
- **Metrics**
- **Logs**

For distributed tracing, OpenTelemetry helps us follow a request as it moves through different microservices.

Example

Client



API Gateway



Order Service



Payment Service



Notification Service

OpenTelemetry can create and propagate tracing information through this entire flow.

Trace and Span

OpenTelemetry uses two important concepts:

Trace

A **trace** represents the complete journey of one request.

Trace ID = ABC-123456

Span

A **span** represents one operation within the trace.

Trace: ABC-123456

|— Gateway Span

|— Order Service Span

|— Payment Service Span

|— Notification Service Span

So:

Trace = complete request

Span = individual operation

Why OpenTelemetry is useful

Imagine Payment Service fails.

Without tracing:

Payment failed.

We don't know why or where the original request came from.

With OpenTelemetry:

Trace ID: ABC-123456

Gateway ✓

Order Service ✓

Payment Service ✗

We can quickly identify the failed service.

Step 9 — Create a Request-Flow Diagram

1. Client

The client starts the request.

For example:

POST /orders

2. API Gateway

The request first reaches the Gateway.

The Gateway creates or receives a Trace/Correlation ID:

Trace ID = ABC-123456

3. Order Service

Gateway sends the request to Order Service.

X-Correlation-ID: ABC-123456

Order Service performs the order operation.

4. Payment Service

Order Service calls Payment Service.

The same tracing information is propagated:

X-Correlation-ID: ABC-123456

Payment Service processes the payment.

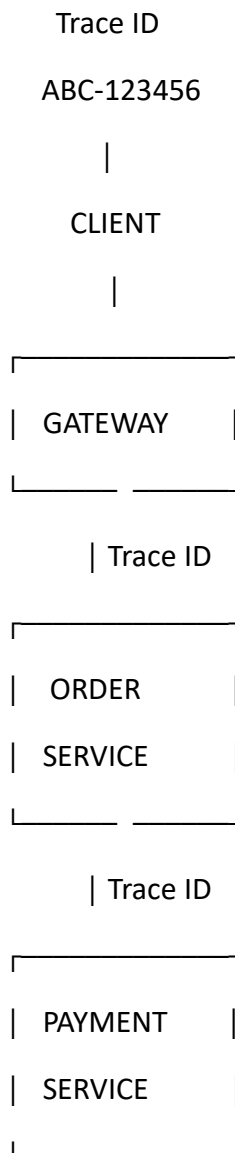
5. Notification Service

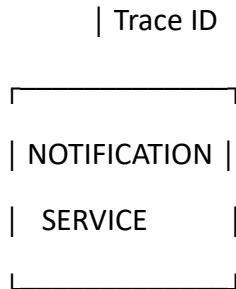
After payment succeeds, the request reaches Notification Service.

Again: X-Correlation-ID: ABC-123456

Notification Service sends the confirmation.

Complete Request Flow





The same trace information follows the request through the services.

Step 10 — Prepare Troubleshooting Documentation

We should document **how to investigate a request when something goes wrong**.

For example:

User reports:

"Order creation failed."

Your troubleshooting process should be:

1. Get Trace/Correlation ID



2. Search Gateway logs



3. Follow the request



4. Check Order Service



5. Check Payment Service



6. Check Notification Service



7. Find the failed service



8. Check error message



9. Check response time



10. Identify root cause

Example Troubleshooting

Suppose the Trace ID is:

ABC-123456

Search logs for:

ABC-123456

You find:

Gateway

ABC-123456 → 200 OK

Order Service

ABC-123456 → 201 Created

Payment Service

ABC-123456 → 503 Service Unavailable

Notification Service

No request received

Now we know:

Gateway ✓

Order Service ✓

Payment Service ✗

Notification —

Root cause

Payment Service was unavailable.